

03 — Train Model 1: XLM-RoBERTa + R-Drop

Chạy thử 3. Input: faidset-processed + augmented-data. **Output:** xlmroberta-rdrop dataset.

Cải tiến so với baseline:

- Dùng `train_aug.csv` (có thêm ~4000 Human mẫu từ back-translation)
- **R-Drop:** mỗi batch forward 2 lần, thêm KL divergence loss → `total_loss = CE_loss + 0.5 × KL_loss`

```
!pip install transformers torch scikit-learn sentencepiece -q
12.2/12.2 MB 98.0 MB/s eta
0:00:00:00:010:01
ERROR: pip's dependency resolver does not currently take into account
all the packages that are installed. This behaviour is the source of
the following dependency conflicts.
dask-cuda 26.2.0 requires cuda-core==0.3.*, but you have cuda-core
1.0.1 which is incompatible.
dask-cuda 26.2.0 requires numba-cuda<0.23.0,>=0.22.1, but you have
numba-cuda 0.30.2 which is incompatible.
distributed-ucxx-cu12 0.48.0 requires numba-
cuda[cu12]<0.23.0,>=0.22.1, but you have numba-cuda 0.30.2 which is
incompatible.
cuml-cu12 26.2.0 requires numba<0.62.0,>=0.60.0, but you have numba
0.65.1 which is incompatible.
cuml-cu12 26.2.0 requires numba-cuda[cu12]<0.23.0,>=0.22.1, but you
have numba-cuda 0.30.2 which is incompatible.
ucxx-cu12 0.48.0 requires numba-cuda[cu12]<0.23.0,>=0.22.1, but you
have numba-cuda 0.30.2 which is incompatible.
cudf-cu12 26.2.1 requires numba<0.62.0,>=0.60.0, but you have numba
0.65.1 which is incompatible.
cudf-cu12 26.2.1 requires numba-cuda[cu12]<0.23.0,>=0.22.2, but you
have numba-cuda 0.30.2 which is incompatible.

import pandas as pd
import numpy as np
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer,
AutoModelForSequenceClassification, get_linear_schedule_with_warmup
from torch.optim import AdamW
from sklearn.metrics import f1_score, classification_report
from sklearn.utils.class_weight import compute_class_weight
from pathlib import Path
import json, os, subprocess
```

```

torch.manual_seed(42)
np.random.seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {device}")

MODEL_NAME = "xlm-roberta-base"
OUTPUT_DIR = "models/xlmroberta_rdrop"
TRAIN_CSV = "/kaggle/input/datasets/cminhnguyndsd/sds/failedset-processed/train.csv"
VAL_CSV = "/kaggle/input/datasets/cminhnguyndsd/sds/failedset-processed/val.csv"
MAX_LEN = 128
BATCH_SIZE = 32
EPOCHS = 3
LR = 2e-5
RDROP_ALPHA = 0.5

Device: cuda

class TextDataset(Dataset):
    def __init__(self, df, tokenizer, max_len):
        self.texts = df["text"].tolist()
        self.labels = df["label"].tolist()
        self.tok = tokenizer
        self.max_len = max_len

    def __len__(self): return len(self.texts)

    def __getitem__(self, idx):
        enc = self.tok(self.texts[idx], max_length=self.max_len,
                        padding="max_length", truncation=True,
return_tensors="pt")
        return {"input_ids": enc["input_ids"].squeeze(),
                "attention_mask": enc["attention_mask"].squeeze(),
                "label": torch.tensor(self.labels[idx],
dtype=torch.long)}

def compute_kl_loss(p_logits, q_logits):
    """Bidirectional KL divergence – trung tâm của R-Drop."""
    p = F.softmax(p_logits, dim=-1)
    q = F.softmax(q_logits, dim=-1)
    loss = (F.kl_div(F.log_softmax(p_logits, dim=-1), q,
reduction="batchmean") +
            F.kl_div(F.log_softmax(q_logits, dim=-1), p,
reduction="batchmean")) / 2
    return loss

```

```
print("Utilities ready")
```

Utilities ready

```
train_df = pd.read_csv(TRAIN_CSV, encoding="utf-8-sig")
val_df    = pd.read_csv(VAL_CSV,    encoding="utf-8-sig")
print(f"Train: {len(train_df):,} | Val: {len(val_df):,}")
print(train_df.groupby(["language", "label"]).size().to_string())
```

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model     =
AutoModelForSequenceClassification.from_pretrained(MODEL_NAME,
num_labels=2).to(device)
```

```
train_loader = DataLoader(TextDataset(train_df, tokenizer, MAX_LEN),
batch_size=BATCH_SIZE, shuffle=True)
val_loader   = DataLoader(TextDataset(val_df,    tokenizer, MAX_LEN),
batch_size=BATCH_SIZE)
```

Class weight – bù cho mất cân bằng EN

```
cw = compute_class_weight("balanced", classes=np.array([0, 1]),
                          y=train_df[train_df["language"]=="en"]
["label"].values)
loss_fn = torch.nn.CrossEntropyLoss(weight=torch.tensor(cw,
dtype=torch.float).to(device))
print(f"Class weights EN: human={cw[0]:.4f}, AI={cw[1]:.4f}")
```

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.

Train: 48,257 | Val: 4,920

language	label	
en	0	14227
	1	14989
vi	0	11984
	1	7057

```
{"model_id": "fa5e7d57a8734060a5d0b206be078d6a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "6adfd4add104f6cbe1992e9c0d0bf57", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "51dbb38883bc49de91ce16da2f743bd9", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "fb7cced1299a42eca0613aelbc604fbd", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f3f9acf8ab74452cbf4acb5d004a7931", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "be749459ca7246f39741d8e3635df3cc", "version_major": 2, "version_minor": 0}
```

XLNetForSequenceClassification LOAD REPORT from: xlm-roberta-base

Key	Status
roberta.pooler.dense.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
roberta.pooler.dense.bias	UNEXPECTED
lm_head.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.dense.bias	MISSING

Notes:

- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING : those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Class weights EN: human=1.0268, AI=0.9746

```
optimizer = AdamW(model.parameters(), lr=LR, weight_decay=0.01)
total_steps = len(train_loader) * EPOCHS
scheduler = get_linear_schedule_with_warmup(optimizer,
int(0.1*total_steps), total_steps)
Path(OUTPUT_DIR).mkdir(parents=True, exist_ok=True)
best_f1 = 0
```

```
for epoch in range(EPOCHS):
```

```
    model.train()
    total_loss = 0
```

```
    for batch in train_loader:
```

```
        optimizer.zero_grad()
        ids = batch["input_ids"].to(device)
        mask = batch["attention_mask"].to(device)
        lbls = batch["label"].to(device)
```

```
        # R-Drop: 2 forward pass với dropout mask khác nhau
```

```
        out1 = model(input_ids=ids, attention_mask=mask)
        out2 = model(input_ids=ids, attention_mask=mask)
```

```
        ce_loss = (loss_fn(out1.logits, lbls) + loss_fn(out2.logits,
```

```

lbls)) / 2
    kl_loss = compute_kl_loss(out1.logits, out2.logits)
    loss = ce_loss + RDR0P_ALPHA * kl_loss

    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    scheduler.step()
    total_loss += loss.item()

model.eval()
preds, trues = [], []
with torch.no_grad():
    for batch in val_loader:
        out = model(input_ids=batch["input_ids"].to(device),
attention_mask=batch["attention_mask"].to(device))
        preds.extend(out.logits.argmax(-1).cpu().numpy())
        trues.extend(batch["label"].numpy())

val_f1 = f1_score(trues, preds, average="macro")
print(f"Epoch {epoch+1}/{EPOCHS} | loss:
{total_loss/len(train_loader):.4f} | val F1: {val_f1:.4f}")
if val_f1 > best_f1:
    best_f1 = val_f1
    model.save_pretrained(OUTPUT_DIR)
    tokenizer.save_pretrained(OUTPUT_DIR)
    print(f"    Saved (F1={best_f1:.4f})")

print(f"\n Done. Best val F1: {best_f1:.4f}")

Epoch 1/3 | loss: 0.2666 | val F1: 0.9154

{"model_id": "c62554b28ba24f169b0ce1149aaa2b79", "version_major": 2, "version_minor": 0}

    Saved (F1=0.9154)
Epoch 2/3 | loss: 0.1157 | val F1: 0.9310

{"model_id": "233a9c1474eb4143a70f8909a3e23c52", "version_major": 2, "version_minor": 0}

    Saved (F1=0.9310)
Epoch 3/3 | loss: 0.0623 | val F1: 0.9229

Done. Best val F1: 0.9310

from tqdm import tqdm

# Load model tốt nhất vừa train
model_best =

```

```

AutoModelForSequenceClassification.from_pretrained(OUTPUT_DIR).to(device).eval()
tok_best = AutoTokenizer.from_pretrained(OUTPUT_DIR)

val_df = pd.read_csv(VAL_CSV, encoding="utf-8-sig")
val_loader_eval = DataLoader(TextDataset(val_df, tok_best, MAX_LEN),
                              batch_size=BATCH_SIZE)

preds, trues = [], []
with torch.no_grad():
    for batch in tqdm(val_loader_eval, desc="Evaluating"):
        out = model_best(
            input_ids=batch["input_ids"].to(device),
            attention_mask=batch["attention_mask"].to(device)
        )
        preds.extend(out.logits.argmax(-1).cpu().numpy())
        trues.extend(batch["label"].numpy())

print(f"\nXLM-RoBERTa val F1: {f1_score(trues, preds,
average='macro'):.4f}")
print(classification_report(trues, preds, target_names=["Human",
"AI"]))

{"model_id": "76aea467af3e4288bbac150e8304b30e", "version_major": 2, "version_minor": 0}

```

Evaluating: 100%|██████████| 154/154 [00:35<00:00, 4.32it/s]

XLM-RoBERTa val F1: 0.9310

	precision	recall	f1-score	support
Human	0.98	0.89	0.93	2470
AI	0.89	0.98	0.93	2450
accuracy			0.93	4920
macro avg	0.93	0.93	0.93	4920
weighted avg	0.93	0.93	0.93	4920

Upload model lên Kaggle Dataset

```

dataset_name = "xlmroberta-rdrop"
kaggle_user = [l.split(":")[1].strip() for l in
                subprocess.run("kaggle config view", shell=True,
capture_output=True, text=True)
                .stdout.split("\n") if "username" in l][0]

with open(f"{OUTPUT_DIR}/dataset-metadata.json", "w") as f:
    json.dump({"title": dataset_name, "id":

```

```
f"{kaggle_user}/{dataset_name}",
    "licenses": [{"name": "CC0-1.0"}]}, f)

check = subprocess.run(f"kaggle datasets list --user {kaggle_user} --
search {dataset_name}",
                        shell=True, capture_output=True, text=True)
cmd = (f"kaggle datasets version -p {OUTPUT_DIR} -m "update"
      if dataset_name in check.stdout
      else f"kaggle datasets create -p {OUTPUT_DIR}")
print(subprocess.run(cmd, shell=True, capture_output=True,
text=True).stdout)
print(f"Done! {kaggle_user}/{dataset_name}")
```

Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.1), please consider upgrading to the latest version
(2.0.2)

Starting upload for file tokenizer.json

Upload successful: tokenizer.json (16MB)

Starting upload for file config.json

Upload successful: config.json (761B)

Starting upload for file tokenizer_config.json

Upload successful: tokenizer_config.json (314B)

Starting upload for file model.safetensors

Upload successful: model.safetensors (1GB)

Dataset version is being created. Please check progress at

<https://www.kaggle.com/datasets/cminhnguyndsd/sds/xlmroberta-rdrop>

Done! cminhnguyndsd/sds/xlmroberta-rdrop